

Reviewer Pack — Real Stable Polynomial Degree Bounds SOS Refutation Threshold...

Entry 03c7b5e2f51b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 04:02:35 UTC

Real Stable Polynomial Degree Bounds SOS Refutation Threshold for 3-XOR

Entry ID: 03c7b5e2f51b Verdict: INCONCLUSIVE Recorded: 2026-05-20 04:02:35 UTC

1. Conjecture

Field A (mathematical branch): Real stable polynomials **Field B** (complexity object): SOS refutation degree for 3-XOR CSPs

Statement:

For a random 3-XOR formula with n variables, the SOS refutation degree required to prove unsatisfiability is at most the degree of the real stable polynomial representing the constraint system, which is $\Theta(\sqrt{n})$ for formulas with clause density ≥ 1.5 .

Rationale (proposer’s reasoning):

Real stable polynomials encode robustness against perturbations, and their degree captures the minimal ‘energy’ needed to violate constraints. SOS hierarchies approximate such energy via semidefinite programming, suggesting a direct link between polynomial stability and refutation complexity.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cb6b749a0c70b311

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def construct_constraint_polynomial(clauses):
        n = len(clauses[0])
        poly = 1
        x = [Fraction(1, 1)] * n
        for clause in clauses:
            linear_form = [random.choice([-1, 1]) for _ in range(n)]
            poly *= (linear_form[0] * x[0] + linear_form[1])
        return poly

    def is_real_stable(poly):
        roots = [complex(root) for root in poly.roots]
        for root in roots:
            if not (root.imag == 0 and root.real >= 0):
                return False
        return True

    def sdp_relaxation(poly, max_iter=100):
        n = len(poly)
        x = [Fraction(1, 1)] * n
        for _ in range(max_iter):
            gradient = [sum(coeff * root ** i for coeff, root in zip(poly, roots)) for i in range(n)]
            step_size = Fraction(1, sum(abs(g) for g in gradient))
            x = [x_i - step_size * grad_i for x_i, grad_i in zip(x, gradient)]
        return max(abs(x_i) for x_i in x)

    n = 40
    clause_density = 1.5
    num_clauses = int(n * clause_density)
    clauses = [[random.choice([-1, 1]) for _ in range(n)] for _ in range(num_clauses)]

    poly = construct_constraint_polynomial(clauses)
    if not is_real_stable(poly):
```

```

    return {
        "metric_name": "sos_refutation_degree",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

sos_refutation_degree = sdp_relaxation(poly)
conjecture_holds = sos_refutation_degree <= math.sqrt(n)

return {
    "metric_name": "sos_refutation_degree",
    "metric_value": sos_refutation_degree,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(result['metric_value'] for result in results)/len(results)}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e67ae697.py", line 77, in <module>
    trial_result = run_trial(seed)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e67ae697.py", line 52, in run_trial
    if not is_real_stable(poly):
    ~~~~~

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e67ae697.py", line 31, in is_real_stabl
    roots = [complex(root) for root in poly.roots]
              ^^^^^^^^^^^
```

AttributeError: 'Fraction' object has no attribute 'roots'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during execution, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix the 'Fraction' object handling in is_real_stable() to enable proper root extraction

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136585
2	propose	ollama_remote	qwen3:8b	0	0	103263
3	preregistration	ollama_remote	qwen3:8b	0	0	31938
4	novelty	ollama_remote	qwen3:8b	0	0	27494
5	novelty	ollama_remote	qwen3:8b	0	0	19753
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12915
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11981
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10058
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9477
10	judge	ollama_remote	qwen3:8b	0	0	22115

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 385579 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/03c7b5e2f51b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/03c7b5e2f51b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/03c7b5e2f51b.tar.gz` (if generated)